

Automação FAQ

a porta de entrada

Começa a série que fecha o módulo: um atendimento automático de dúvidas, em que o aluno pergunta, o Make busca na FAQ, a IA sugere uma resposta e a equipe aprova antes de enviar. Nesta primeira parte, construímos **a entrada**: uma tabela de **Atendimentos** no Airtable, um **webhook** protegido por token, o registro de cada pergunta e a **resposta imediata** ao site.

Arquitetura da série

Tabela Atendimentos

Custom webhook

Data structure

Token no cabeçalho

Webhook response

Formação Claude Code 2026 · Baseado em [help.make.com](https://help.make.com/en/articles/11411-webhooks) (Webhooks, Operations, Scenario settings) e [apps.make.com](https://apps.make.com/en/articles/11411-webhooks) (Webhooks, Airtable), consultados em outubro de 2026 · Telas: reprodução ilustrativa

As quatro partes *da automação FAQ*

AULA 07 • ENTRADA

Webhook recebe a pergunta, registra e responde ao site.



AULA 08 • BUSCA + IA

A FAQ publicada vai para a IA, que sugere uma resposta em JSON.



AULA 09 • DECISÃO

Router: encontrou × não encontrou; tratamento de erros.



AULA 10 • ENVIO

Só respostas **aprovadas** por uma pessoa saem para o aluno.



RESULTADO

Atendimento rápido, sem resposta inventada chegando ao aluno.

DOIS CENÁRIOS

FAQ • Receber pergunta (aulas 07–09) e **FAQ • Enviar respostas aprovadas** (aula 10). No Free, são os seus 2 cenários ativos.

O AIRTABLE NO CENTRO

Tudo fica registrado lá: a pergunta, a sugestão da IA, a decisão do revisor e o envio.

HUMANO NO MEIO

Mesma regra do User approval (módulo 31): a IA sugere, uma pessoa aprova.

A tabela *Atendimentos*

CAMPO	TIPO
Pergunta recebida (primário)	Long text
Email do aluno	Email
Origem	Single select: Site, Formulário, E-mail
Status	Single select: Novo, Sugerida, Sem resposta, Aprovado, Enviado, Descartado
Resposta sugerida	Long text (IA, aula 08)
Resposta final	Long text (revisor, aula 10)
FAQ usada	Link → Perguntas
Recebido em / Enviado em	Created time / Date

POR QUE UMA TABELA SEPARADA

Perguntas é a FAQ curada: pública, sem dados pessoais. **Atendimentos** guarda o que chega, com o e-mail do aluno. Separar permite dar acessos diferentes e **apagar dados pessoais** depois de um tempo (aula 10).

O e-mail só existe para responder. Diga isso no formulário do site e defina por quanto tempo ele fica guardado (LGPD: finalidade e necessidade).

Lembre do limite do Free: **1.000 registros por base**, somando Perguntas, Categorias, Aulas e Atendimentos.

Por que *webhook* agora

	AULA 06 (CHECAGEM)	AULA 07 (WEBHOOK)
Quando roda	4 vezes por dia	Na hora em que a pergunta chega
Créditos sem pergunta	1 por checagem	Zero
De onde vem	Só do Airtable	De qualquer lugar: site, formulário, outro sistema
Resposta ao usuário	Nenhuma	Imediata ("recebemos sua pergunta")

COMO FUNCIONA

O Make cria um **endereço**. Quem envia um POST para ele entrega um bundle ao cenário. Segundo a central de ajuda, cada webhook tem a sua **fila**: as requisições ficam guardadas até serem processadas.

Sem o módulo de resposta, o Make devolve `200 Accepted` ; com a fila cheia, `400` ; com excesso de chamadas, `429 Too many requests` . O site precisa tratar esses casos.

O webhook *e a estrutura de dados*

1 Novo cenário: **FAQ · Receber pergunta**. Primeiro módulo: **Webhooks · Custom webhook**.

2 Add webhook, nome "faq-entrada". Ligue **Get request headers** (para ler o token).

3 Em **Data structure**, crie a estrutura ao lado.

4 Copie a URL e guarde como segredo.

```
// Data structure · "faq-pergunta"
{
  "pergunta": Text    (obrigatório)
  "email":    Text    (obrigatório)
  "origem":   Text    (site | formulario)
}
```

Com a estrutura definida, o Make **valida** o que chega e os campos aparecem no painel de mapeamento com nome e tipo. Requisições fora do formato são rejeitadas.

Outra opção: clicar em **Redetermine data structure** e enviar uma requisição de exemplo; o Make deduz os campos.

Trancar *a porta*

A URL do webhook é pública por natureza: quem a descobrir pode mandar perguntas falsas, gastar seus créditos e encher a base. Uma proteção simples: exigir um **token secreto** num cabeçalho e filtrar.

```
// Filtro logo após o webhook - "Token confere"
Condição
1. Headers[] → x-faq-token
   Text operators: Equal to (case sensitive)
   <token guardado num lugar seguro>
```

- ✓ O token fica **no servidor do site** (por exemplo, numa função de back-end ou edge function), nunca no JavaScript do navegador
- ✓ Requisição sem token não passa do filtro: nada é gravado
- ✓ Troque o token se desconfiar de vazamento
- ✓ Limite o tamanho da pergunta no site (ex.: 1.000 caracteres)

O gatilho ainda consome 1 crédito por requisição, mesmo barrada. Para ataques em volume, a proteção precisa estar também **antes** do Make: captcha e limite de envios no site.

Registrar *o atendimento*

AIRTABLE • CREATE A RECORD		VALOR
Table	Atendimentos	
Pergunta recebida	{{trim(1.pergunta)}}	
Email do aluno	{{lower(trim(1.email))}}	
Origem	{{ifempty(1.origem; "Site")}}	
Status	Novo	

REGISTRAR PRIMEIRO

A pergunta é gravada **antes** de qualquer IA. Se algo falhar depois, ela já está no Airtable e ninguém fica sem resposta.

O ID DO ATENDIMENTO

A saída deste módulo traz o **ID** do registro criado (**rec...**). As próximas aulas usam esse ID para atualizar o mesmo atendimento.

Pequenas limpezas com funções (**trim** , **lower**) evitam e-mails duplicados por causa de maiúsculas ou espaços.

Responder *ao site*

```
// Webhooks - Webhook response
Status: 200
Body:
{
  "ok": true,
  "mensagem": "Recebemos sua pergunta. A resposta chega por e-mail."
}
Custom headers:
  Content-Type: application/json
```

A central de ajuda recomenda ajustar o **Content-Type** ao formato do corpo: com JSON, `application/json` .

POR QUE RESPONDER JÁ

A IA e a revisão podem levar minutos ou horas. O aluno precisa saber, na hora, que a pergunta chegou e como a resposta vai chegar.

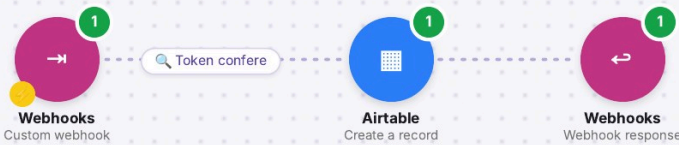
E SE O TOKEN NÃO CONFERIR?

Adicione uma segunda rota (com Router) que responde `401` e `{ "ok": false }` . Sem isso, o site recebe o "Accepted" padrão e acha que deu certo.

A resposta não deve conter o ID interno nem dados de outros alunos. Só o necessário para o site mostrar a mensagem.

Testar *pelo terminal*

FAQ · Receber pergunta ✓ The scenario was run successfully Credits: 3



Webhooks · Custom webhook Operation 1 · Output

```
pergunta: "Como conecto o Lovable ao Supabase?"
email: "aluna@exemplo.com"
origem: "site"
__IMTHEADERS__ (Array)
x-faq-token: "....."
content-type: "application/json"
```

```
# Teste pelo terminal (troque a URL e o token)
$ curl -X POST "https://hook.us1.make.com/xxxxxxx" \
  -H "Content-Type: application/json" -H "X-FAQ-Token: $FAQ_TOKEN" \
  -d '{"pergunta":"Como conecto o Lovable ao Supabase?","email":"aluna@exemplo.com","origem":"site"}'
{"ok": true, "mensagem": "Recebemos sua pergunta. A resposta chega por e-mail."}
```

▶ Run once ⌚ Immediately

Run once esperando a requisição de teste · reprodução ilustrativa

1 · RUN ONCE

O cenário fica esperando dados no webhook.

2 · ENVIE COM CURL

Ou com o formulário do site. Use seu próprio e-mail no teste.

3 · CONFIRA

Resposta JSON no terminal e um registro "Novo" em Atendimentos.

Os casos *que precisam funcionar*

TESTE	ESPERADO
Pergunta válida, com token	200, registro "Novo" criado
Sem o cabeçalho de token	Barrada no filtro; nada gravado (e, com a rota extra, 401)
Token errado	Igual ao anterior
Sem o campo <code>pergunta</code>	Rejeitada pela estrutura de dados
E-mail com maiúsculas e espaços	Gravado limpo, em minúsculas
Texto com "ignore suas regras..."	Gravado como texto comum; é só uma pergunta (a defesa vem na aula 08)
Duas perguntas ao mesmo tempo	Dois registros; com <i>Process data in order</i> , uma de cada vez

No menu **Webhooks**, o log de cada requisição mostra cabeçalhos, corpo e a resposta enviada. É o primeiro lugar para olhar quando o site diz que "não chegou".

Ligar *o site*

```
// No back-end do site (ex.: função serverless)
const r = await fetch(process.env.MAKE_FAQ_WEBHOOK, {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "X-FAQ-Token": process.env.FAQ_TOKEN,
  },
  body: JSON.stringify({
    pergunta: texto.slice(0, 1000),
    email,
    origem: "site",
  }),
});
if (!r.ok) /* mostrar "tente de novo em instantes" */
```

ONDE FICA O CÓDIGO

No back-end: uma edge function do Supabase (módulo 28), uma rota de API ou uma função serverless. URL e token vêm de **variáveis de ambiente**.

Nunca chame o webhook direto do navegador com o token no código: qualquer visitante veria o token ao abrir as ferramentas do desenvolvedor.

Um site feito no Lovable (módulo 27) com Supabase (módulo 28) é um ótimo "rosto" para esta FAQ.

Quanto custa *a entrada*

POR PERGUNTA	CRÉDITOS MAKE	CHAMADAS AIRTABLE
Webhook	1	—
Filtro	0	—
Create a record	1	1
Webhook response	1	—
Total	3	1

E AS CHECAGENS?

Zero. Com webhook, o cenário só roda quando chega pergunta. É a grande economia em relação à aula 06.

As aulas 08 e 09 somam busca, IA, atualização e avisos. Na aula 10 fazemos a conta completa por pergunta e por mês.

Erros comuns *na entrada*

Webhook sem token. Qualquer um envia perguntas falsas e gasta seus créditos.

Token no JavaScript do navegador. É o mesmo que publicá-lo.

Chamar a IA antes de gravar. Se a IA falhar, a pergunta some.

Sem Webhook response. O site recebe "Accepted" até quando algo deu errado.

E-mail do aluno na FAQ pública. Dados pessoais ficam só em Atendimentos.

Testar com e-mails de verdade de alunos. Use o seu.

Teste o que *ficou*

1. Por que separar Atendimentos de Perguntas?

A) Por estética

B) Para manter a FAQ curada sem dados pessoais e poder apagar e-mails depois

C) Porque o Airtable exige

D) Para economizar créditos

B. Dados pessoais no lugar certo.

2. Onde deve ficar o token do webhook?

A) No JavaScript da página

B) Em variável de ambiente do back-end do site

C) Na URL

D) No Airtable

B. Nunca no navegador.

3. Sem o módulo Webhook response, o que o site recebe?

A) Nada

B) `200 Accepted`, mesmo que algo dê errado depois

C) A resposta da IA

D) Um erro

B. Por isso respondemos explicitamente.

4. Por que gravar a pergunta antes de chamar a IA?

A) Fica mais bonito

B) Se a IA falhar, a pergunta não se perde

C) A IA exige

D) Custa menos

B. Registrar primeiro.

○ RECAPITULANDO

A porta está aberta *e trancada*

Desenhemos a série em quatro partes e criamos a tabela Atendimentos, separada da FAQ curada por causa dos dados pessoais. Montamos o webhook com estrutura de dados, um filtro de token no cabeçalho, o registro do atendimento com limpeza de dados e uma resposta JSON imediata ao site. Testamos com curl os casos que importam e vimos como ligar o back-end do site sem expor o token. Custo: 3 créditos por pergunta e nenhuma checagem vazia.

Atendimentos

Custom webhook

Data structure

X-FAQ-Token

Create a record

Webhook response 200

Próxima aula → Automação FAQ II

Fontes: help.make.com (Webhooks, Operations e Scenario settings) e apps.make.com (Webhooks e Airtable), consultados em outubro de 2026. URLs e tokens nos exemplos são fictícios. As telas são reproduções ilustrativas feitas para o curso.